

EL403 – Assignment 1

Karthik Maddimsetti (202304006) and Het Patel (202304033), *Dhirubhai Ambani University, Gandhinagar.*

About the document — In this document, we have presented our understanding, analysis and conclusions on the given exercises, with all the discussions relating them to VLSI-DSP Circuits perspective by including about scalability, trade-offs and few further applications of the circuits. The simulation results are based on the transition dependent delays taken from Cadence designs of basic gates made during the last semester.

COMMON METHODOLOGY

All circuits described in this report were modeled in structural Verilog using basic logic gates whose delays were obtained from full-custom designs created and simulated in Cadence Virtuoso. The gate delays were *transition-dependent*, with separate rise and fall values. These delays were incorporated into the Verilog gate models using **specify** blocks, to ensure that the timing behavior in simulation closely matched the transistor-level implementation. Functional and timing simulations were performed in Xilinx Vivado, to observe delay effects. The same methodology was applied consistently to all adders and multiplier discussed in this report.

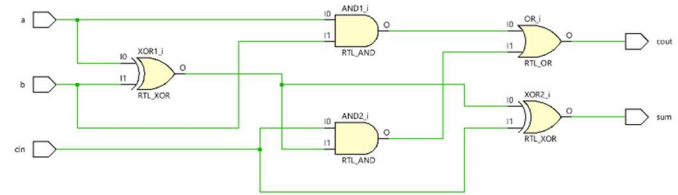
The following are the values of the delays obtained from the Cadence simulations:

Table-1:

Gate	Input Transition	Output Transition	Delay (ns)
NAND2	A=1, B: 0→1	1→0	0.588
	A=1, B: 1→0	0→1	0.915
	B=1, A: 0→1	1→0	0.677
	B=1, A: 1→0	0→1	1.064
NOR2	A=0, B: 0→1	1→0	0.076
	A=0, B: 1→0	0→1	0.275
	B=0, A: 0→1	1→0	0.426
	B=0, A: 1→0	0→1	0.087
XOR2	A=0, B: 0→1	0→1	0.075
	A=0, B: 1→0	1→0	0.855
	A=1, B: 0→1	1→0	0.156
	A=1, B: 1→0	0→1	0.534
	B=0, A: 0→1	0→1	0.086
	B=0, A: 1→0	1→0	0.860
	B=1, A: 0→1	1→0	0.013

Gate	Input Transition	Output Transition	Delay (ns)
	B=1, A: 1→0	0→1	0.381
INV	A: 0→1	1→0	0.372
	A: 1→0	0→1	0.597

I. FULL ADDER



First, we have implemented a full adder using the basic gates because it is the basic building block of all the following circuits.

The following are the by hand-analysis and conclusions that helped us in estimating the worst-case delay and the corresponding input combinations:

Table-2:

Carry propagation:

A	B	Cin	Cout
1	0	0 or 1	cin
0	1	0 or 1	cin

Carry generation:

A	B	Cin	Cout
1	1	0 or 1	1

$$T_{sum} = \max(T_{xor}, T_{cin}) + T_{xor2} \quad (1)$$

If carry is generated, $T_{cout} = T_{AND1} + T_{OR}$ (2)

If carry is propagated, $T_{cout} = \max(T_{xor1}, T_{ci}) + T_{AND2} + T_{OR}$ (3)

From (2), (3) and Table-2, we can understand that, when carry is generated, it traverses through only 2 gates (AND1 and OR) but when it is propagated it will traverse through 3 gates (XOR, AND2 and OR). So generated *cout* would be faster than propagated *cout*.

II. RIPPLE CARRY ADDER

A Ripple Carry Adder (RCA) is the simplest form of a multi-bit adder, built by cascading n full adders so that the carry

“ripples” from least significant bit (LSB) to the most significant bit (MSB).

We have implemented an *n-bit Ripple Carry Adder*. (*n* is the width of the operands)

A. Inputs:

- A[n-1:0], B[n-1:0]: Operands
- Cin : Initial carry-in

B. Outputs:

- Sum[n-1:0]: Sum bits
- Cout: Final carry-out

C. Delay Characteristics:

- *Slowest Path:*

Let T_{carry} be the carry-out delay of a single Full Adder (FA), and T_{sum} be the sum delay of a single FA.

The worst-case carry-out delay is given by

$$n \times T_{carry} \quad (4)$$

The worst-case sum delay for MSB is given by

$$(n - 1) \times T_{carry} + T_{sum} \quad (5)$$

- *Slowest Path Input Combination:*

As we discussed in I for full adders, we get the worst-case delay when all the carries between the full adders in the RCA are propagated, this happens when the input combination is as follows:

$$11111\dots1 \text{ (n 1s)} + 00\dots001 \text{ (n-1 0s and LSB = 1)} + (\text{Cin} = 1)$$

This combination causes the carry generated at the LSB to ripple and propagate through all the full adders creating the maximum delay path.

D. Results

Worst case delay for Cout = 18.539 ns

Worst case delay for Sum = 16.740 ns

Simulation results are shown in Fig.1

E. Notes:

Scalability: Worst case delay increases linearly with *n*. For *n* = 8; Maximum value that can be represented using the sum bits i.e., Sum of maximum inputs, 255 + 255 = 510 will require 9 bits.

Overflow: Cout becomes 1 indicating the overflow in the sum.

Carry Propagate Adder (CPA): RCA is a CPA because each sum bit depends on the previous stage's carry-out, which must be computed before the next sum bit is determined.

Application: It is used in circuits like multipliers to propagate the carry in the last stage to compute the final product.

(As shown in question 2 and 3)

Trade-off: The RCA uses simple logic and less area at the cost of speed as compared to the other CPAs.

III. CARRY LOOK-AHEAD ADDER AS CARRY PROPAGATE ADDER

A Carry Look-Ahead Adder (CLA) is a CPA that speeds up addition by computing carry bits in parallel using generate and propagate logic.

For each bit *i*:

$$(\text{Generate}) G_i = A_i \cdot B_i \quad (5)$$

$$(\text{Propagate}) P_i = A_i \oplus B_i \quad (6)$$

$$C_{i+1} = G_i + (P_i \cdot C_i) \quad (7)$$

We implemented an 8-bit CLA using two 4-bit CLA blocks because 8-bit CLA would require gates with high fan-out to produce generate and propagate signals which would make the circuit slower.

- The carry-out from the first (lower 4-bit) CLA is connected to the carry-in of the second (upper 4-bit) CLA.
- The most significant 4 bits of A and B go to the second CLA, and the least significant 4 bits go to the first CLA.

A. Inputs and Outputs

Same as the RCA mentioned in I.

B. Delay Characteristics:

The worst-case delay for the MSB sum is:

$$T_{Sum(MSB)} = T_{(G,P)} + T_{CarryLogic} + T_{Sum(XOR)} + T_{Cout}$$

Where,

$T_{(G,P)}$ is time to compute generate and propagate signals for the MSB block

$T_{CarryLogic}$ is time to resolve carries inside the CLA block

$T_{Sum(XOR)}$ is time for XOR gate to produce final sum bit

T_{Cout} is final carry-out delay

C. Results

For the same worst-case input combination of RCA in II, (i.e., 8'b11111111 + 8'b00000001 + (Cin = 1)), we get the following delays,

- worst case delay for Cout = 9.306 ns
 - worst case delay for Sum = 8.760 ns,
- which are less than the delays of the RCA.

Simulation results are shown in Fig.2

D. Notes

Scalability: This 8-bit CLA is faster than an 8-bit RCA because each 4-bit block computes its carries in parallel, avoiding the full ripple effect. The delay scales almost logarithmically with bit-width rather than linearly.

Trade-off (Compared to RCA): Faster sum computation for higher bit-widths but needs more area and number of gates (or gates with higher fan-out) due to additional generate/propagate logic.

IV. CARRY SAVE ADDER

A Carry-Save Adder (CSA) belongs to a different class of adders than Carry-Propagate Adders (CPAs). In a CSA, the carry generated at each bit position is not immediately propagated to the next more significant bit. Instead, the carry and sum are stored separately, and the carry propagation is postponed until the final stage, where a CPA produces the final result.

First, we have implemented a CSA for 2 operands, each of 8-bit width, with carry propagation done using an RCA in the final stage.

A. Inputs: $A [7:0], B [7:0], Cin$

B. Outputs: Sum $[8:0], Cout$

Note that, the sum output is 1 bit wider than the operands to accommodate the shifted carry outputs before final addition.

C. Delay Characteristics:

For adding 2 operands, the CSA stage itself has the delay of a single full adder T_{FA} because all sum and carry bits are computed in parallel. The total output delay is:

$$T_{CSA} = T_{FA} + T_{RCA}$$

where T_{RCA} depends on operand width (8 bits in this case). T_{RCA} is the maximum output delay of the RCA.

Worst-Case Delay: Worst case delay occurs when both T_{FA} and T_{RCA} are maximum i.e., carry should be propagated in both the full adder and the RCA.

Worst-Case Input Combination for Maximum Value of Sum:
 $8'b11111111 + 8'b11111111 + 1'b1 = 9'b11111111$,
 which forces carry generation through all bits of the RCA. The 9-bit sum can be accommodated using this system because of the extra bit in the Sum $[8:0]$ without causing an overflow to the Cout bit.

D. Extending to 3-Operand Sum

When the CSA adder is extended to the add 3 operands of 8-bit, the maximum possible sum would be,

$$8'b11111111 + 8'b11111111 + 8'b11111111 + 1'b1 = 10'b1011111110,$$

which will require a 10-bit sum will cause an overflow to the cout bit, which must be handled separately by the system.

E. Results

Worst case delay for Cout = 20.463 ns

Worst case delay for Sum = 18.294 ns

Simulation results are shown in Fig.3

F. Notes

Scalability: Faster multi-operand adders can be built by cascading CSAs. For k operands, $(k-2)$ CSAs and 1 CPA will be needed. Then,

$$T_{SUM} = (k - 2)T_{CSA} + T_{CPA}, \quad \text{where } T_{CPA} = T_{FA}.$$

Application: Multipliers are optimized and modularized using CSA trees. (8-bit x 8-bit Array multiplier using a 7-bit CSAs is shown in IV).

V. MULTIPLIER

From the notion that *multiplication is essentially repeated addition*, and noting that the operands to the adders are the partial products of the input bits, we have implemented the array multiplier by cascading a chain of Carry-Save Adders (CSAs) followed by a final Carry-Propagate Adder (CPA), here implemented as a Ripple Carry Adder (RCA).

This process can be better understood through the example figure¹ of a 5-bit x 5-bit unsigned multiplication shown below. We have extended the same principle to an 8-bit x 8-bit case.

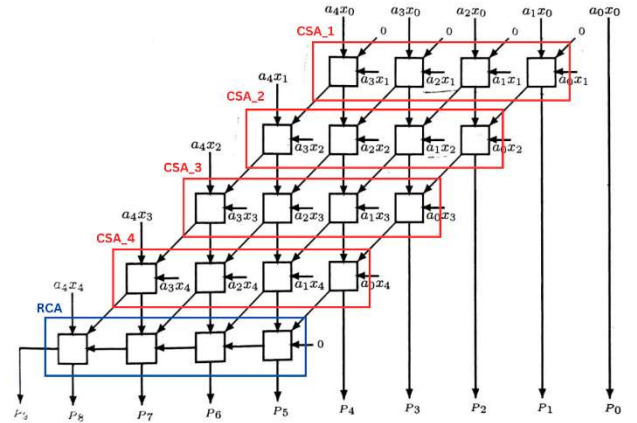


Fig. A: 5-bit x 5-bit unsigned Array Multiplier

We used 7 7-bit CSAs, 1 7-bit RCAs, 65 AND gates (for partial products) to implement the multiplier.

A. Inputs: $A [7:0], B [7:0]$

B. Outputs: $P [15:0]$

C. Delay Characteristics

The total delay of this multiplier is determined by: The linear dependence on the number of CSA stages and the worst-case delay of the final CPA and given by,

$$T_{max} = T_{AND} + 7T_{CSA} + 8T_{carry}$$

Where T_{AND} is the propagation delay of the AND gates used for partial product generation,

T_{CSA} is the delay of one stage of CSA

¹ Figure is taken from Computer Arithmetic Algorithms, 2nd Edition by Israel Koren

T_{carry} is the carry-out delay of FA used in RCA

Worst case delay occurs when them P[14] i.e., most significant sum bit occurs last due to the carry propagation throughout the RCA, if carry generation happens in between the adders in the RCA, it won't be the worst case.

D. Results

The following input combination gives the worst-case delay for the above-mentioned reason,
 $A = 8'b110001111$ and $B = 8'b11110111$
 Worst product delay for product = 31.339 ns
 Simulation results are shown in Fig.4

VI. PIPELINE STAGE WITH NO LOGIC

In the circuit, two register are cascaded (Q1 to D2) without any combinational logic in between, with same clock given both the registers.

Assuming ideal clock (no skew) if,

$T_{clk,min}$ is the minimum time period of the clock cycle,

$T_{cq,max}$ is the maximum clock to q delay of the register,

t_{setup} is the setup time of the register,

t_{hold} is the hold time of the register and

t_{cont} is the contamination delay of the register,

T_{comb} is the propagation delay of the combination circuit between the registers, for a reliable circuit,

$$T_{clk,min} \geq T_{cq,max} + T_{comb,max} + t_{setup}$$

$$\text{and, } t_{cont} + T_{comb,min} \geq t_{hold} \text{ (Hold condition)}$$

In our case, since there is no logic, $T_{comb} \approx 0$, so

$$T_{clk,min} \geq T_{cq,max} + t_{setup}$$

$$t_{cont} \geq t_{hold}$$

This reduces the minimum clock period and increases the maximum operating frequency. But, since $T_{comb,min}$ is extremely small hold checks become critical for reliability.

Relation to DSP circuits: As discussed in the lecture, the maximum operating frequency of the clock can be increased by pipelining the system to increase the *throughput* (the rate at which the output is produced). To do so, registers are inserted between the stages of the combinational circuits (say, multiplier). But from the above discussion, we understand that reducing the combinational circuit's slowest path might expose the system to hold time violations. Hence, one must take care of the hold violation condition when pipelining a high-speed combinational circuit.

Few ways to resolve the hold violation issues and their trade-offs:

- By adding a delay element in the path (fixes hold without much affect on the clock but, area and power increases)
- Using a slower flipflop for the source register (no extra components needed but increases $T_{clk,min}$)

- Inserting a lookup latch to hold the data for the destination register. (increases area and power consumption)

VII. CONCLUSION

Through this assignment, we have explored the implementation trade-offs involved in different arithmetic units and in pipelining DSP circuits. We also learned to model transition dependent timing modules in Verilog, enabling more realistic simulations instead of assuming transition independent delays. The discussion on pipeline stages proved the vulnerabilities of high speed circuits and trade-off between reliability and throughput.

APPENDIX



Fig.1: Worst case delay simulation of 8-bit RCA



Fig.2: Worst case delay simulation of 8-bit CLA



Fig.3: Worst case delay simulation of 8-bit CSA

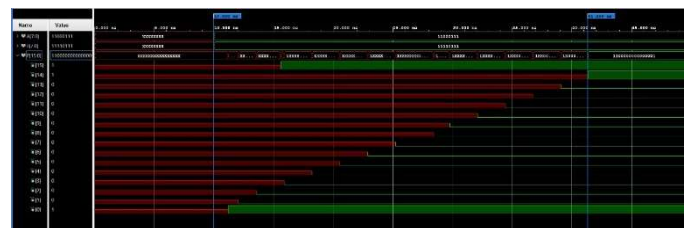


Fig.4: Worst case delay simulation of 8-bit Array Multiplier

REFERENCES

- [1] Used Xilinx Vivado. 2025.1 version for Verilog timing simulations and timing.
- [2] Used to Cadence Virtuoso to model and simulate the basic gates.
- [3] Fig. of 5-bit x 5-bit Array multiplier from Chapter 6 – High-Speed Multiplication of Computer Arithmetic Algorithms by Israel Koren.